

Tarea 2: Generación automática de analizadores léxicos y sintácticos

Oct 5, 2026

Danny Wu

A01286362

Profesora: Elda Quiroga

Github: <https://github.com/dannywu1101/Tarea2LexicoSintaxis->

1. Introducción

Un generador de analizadores recibe una especificación formal y produce automáticamente el código del analizador léxico (*scanner*) y del sintáctico (*parser*) [1]. El mini-proyecto se desarrollará en **C++**, por lo que se analizan tres herramientas que generan o se integran con C++: Lex & Yacc, ANTLR 4 y Boost.Spirit X3.

- **Analizador léxico:** agrupa los caracteres en *tokens* usando expresiones regulares, que se convierten en un autómata finito determinista (AFD).
- **Analizador sintáctico:** verifica que la secuencia de tokens cumpla una gramática libre de contexto (BNF/EBNF). Puede ser ascendente (LR, LALR) o descendente (LL, descenso recursivo).

2. Lex & Yacc

Lex (Lesk y Schmidt, Bell Labs, 1975) genera el scanner y Yacc (Stephen C. Johnson, Bell Labs, 1975) genera el parser; el parser de Yacc llama a la función `yylex()` que produce Lex [2][3]. Ambas son utilidades estándar de POSIX [4][5].

Criterio	Descripción
Plataforma	UNIX/Linux, macOS, BSD; en Windows vía WSL, Cygwin o MSYS2
Lenguaje base	C. El código generado (<code>lex.yy.c</code> , <code>y.tab.c</code> , <code>y.tab.h</code>) también compila con <code>g++</code> , así que se puede usar desde C++ [4][5]
Formato de entrada	Archivos <code>.l</code> y <code>.y</code> con tres secciones separadas por <code>%</code> : definiciones, reglas y código de usuario [2][3]
Tipo de ejecución	Por lotes: la herramienta genera código fuente, que luego se compila a un ejecutable nativo
Licencia y costo	Gratis. Las versiones originales eran parte del UNIX de AT&T; hoy se usan Flex (BSD) [8][17] y GNU Bison (GPL con excepción que permite usar el parser generado en software no libre) [7]
Teoría	Lex: expresiones regulares → AFD, regla de la coincidencia más larga [2][4]. Yacc: gramáticas

Criterio	Descripción
	LALR(1) con desambiguación por precedencia (%left, %right) [3]
Interfaz	Línea de comandos; yacc -v genera y.output con los estados y conflictos [5]

Inyección de código. El código del usuario se copia tal cual al archivo generado:

1. Bloque %{ ... %} en las definiciones: #include, variables y prototipos.
2. Acciones { ... } junto a cada patrón (Lex) o producción (Yacc). En Yacc, \$\$ es el valor de la regla y \$1...\$n los de sus símbolos.
3. Tercera sección (después del segundo %): main(), yyerror() y funciones auxiliares.

Para C++ basta con compilar los archivos generados con g++; dentro de las acciones se puede usar std::string, std::map, etc. Flex y Bison además ofrecen modos nativos de C++ (%option c++ y %language "c++") [7][8]. Referencia clásica: Levine, Mason y Brown [6].

3. ANTLR 4 (destino C++)

ANTLR (*ANother Tool for Language Recognition*), de Terence Parr, genera el lexer y el parser a partir de un solo archivo de gramática y construye automáticamente el árbol sintáctico, junto con clases *listener* y *visitor* para recorrerlo [9][13].

Criterio	Descripción
Plataforma	Windows, Linux y macOS. La herramienta generadora está escrita en Java (necesita una JVM solo para generar) [9]
Lenguaje base	Genera C++ con antlr4 -Dlanguage=Cpp Expr.g4; el código generado se enlaza con el runtime de C++ (antlr4-runtime.h), que requiere C++17 y se compila con CMake o Visual Studio [12]. También genera Java, C#, Python, JavaScript, Go, Swift, etc. [9]
Formato de entrada	Archivo .g4 en notación EBNF. Reglas del parser en minúscula, reglas del lexer en MAYÚSCULA; puede ser una gramática combinada o separada en lexer y parser [11]
Tipo de ejecución	Por lotes: genera ExprLexer.cpp/.h, ExprParser.cpp/.h y las clases listener/visitor, que se compilan con el proyecto
Licencia y costo	BSD de 3 cláusulas, gratis incluso para uso comercial [9]

Criterio	Descripción
Teoría	Lexer basado en expresiones regulares (coincidencia más larga). Parser ALL(*) (<i>Adaptive LL(*)</i>): descendente, con anticipación adaptativa calculada en tiempo de ejecución; acepta recursión izquierda directa reescribiéndola automáticamente [10]
Interfaz	Línea de comandos, más plugins para IntelliJ, VS Code y Eclipse que muestran el árbol sintáctico y diagramas de la gramática

Ejemplo de gramática (la precedencia la da el orden de las alternativas):

```
grammar Expr;
prog : expr EOF ;
expr : expr ('*' | '/') expr    # MulDiv
      | expr ('+' | '-') expr   # AddSub
      | INT                    # Int
      | '(' expr ')'            # Parens ;
INT  : [0-9]+ ;
WS   : [ \t\r\n]+ -> skip ;
```

Inyección de código. Hay dos formas:

1. **Visitor/Listener (recomendada):** se crea una clase C++ que hereda de `ExprBaseVisitor` y se sobrescriben métodos como `visitAddSub(ExprParser::AddSubContext *ctx)`. La gramática queda limpia.
2. **Código incrustado en la gramática,** en C++ y entre llaves: `@header { ... }` para `#include`, `@members { ... }` para atributos y métodos de la clase, acciones `{ ... }` dentro de las reglas y predicados semánticos `{ condición }?` [11].

4. Boost.Spirit X3

Spirit X3 (Joel de Guzman y Hartmut Kaiser) es parte de las bibliotecas Boost de C++ [14]. A diferencia de las otras dos, **no genera archivos:** la gramática se escribe directamente en código C++ usando sobrecarga de operadores, y el compilador de C++ construye el parser mediante plantillas (*expression templates*).

Criterio	Descripción
Plataforma	Cualquier sistema con un compilador moderno de C++ (GCC, Clang, MSVC)
Lenguaje base	C++ puro; biblioteca <i>header-only</i> (solo se incluye <code><boost/spirit/home/x3.hpp></code> , no hay que compilar ni enlazar nada aparte). Desde Boost 1.81 usa características de C++17 [14]

Criterio	Descripción
Formato de entrada	No hay archivo de gramática: las reglas son objetos C++. Operadores: >> secuencia, alternativa, * cero o más, + uno o más, - opcional [14]
Tipo de ejecución	El parser se genera en tiempo de compilación al compilar el programa; no hay paso previo de generación
Licencia y costo	Boost Software License 1.0, gratis y apta para uso comercial [14][15]
Teoría	Gramáticas de expresiones de análisis (PEG) [16]: descenso recursivo con <i>backtracking</i> y alternativa ordenada (gana la primera opción que coincide). Normalmente no usa un lexer separado (<i>scannerless</i>); los espacios se saltan con un <i>skipper</i> como <code>x3::space</code> . No acepta recursión izquierda, así que se escribe <code>expr = term >> *('+' >> term)</code>
Interfaz	API de C++ (biblioteca); no tiene línea de comandos ni interfaz gráfica

Ejemplo de una regla que suma enteros separados por +:

```
namespace x3 = boost::spirit::x3;
int total = 0;
auto sumar = [&](auto& ctx) { total += x3::_attr(ctx); };
auto regla = x3::int_[sumar] >> *('+' >> x3::int_[sumar]);
bool ok = x3::phrase_parse(ini, fin, regla, x3::space);
```

Inyección de código. Es la más directa de las tres, porque la gramática ya es C++:

1. **Acciones semánticas** `parser[funcion]`: cualquier lambda o función que recibe el contexto; con `_attr(ctx)` se lee el valor reconocido y con `_val(ctx)` se asigna el valor de la regla [14].
2. **Atributos automáticos**: el resultado del análisis puede guardarse directamente en `std::vector`, `std::string` o en `struct` propias (adaptadas con `Boost.Fusion`) para construir el AST [14].

5. Comparativa

Criterio	Lex & Yacc	ANTLR 4	Boost.Spirit X3
Lenguaje generado	C (compila con g++)	C++ y otros 9 lenguajes	C++ (no genera archivos)
Entrada	<code>.l + .y</code>	<code>.g4</code>	Código C++

Criterio	Lex & Yacc	ANTLR 4	Boost.Spirit X3
Ejecución	Genera código antes de compilar	Genera código antes de compilar (requiere Java)	Se arma al compilar el programa
Método sintáctico	LALR(1), ascendente	ALL(*), descendente	PEG, descendente con backtracking
Lexer	Separado (Lex)	Integrado en la gramática	Sin lexer separado (skipper)
Licencia	Flex: BSD; Bison: GPL con excepción	BSD	Boost Software License
Interfaz	CLI	CLI + plugins de IDE	API de C++
Inyección de código	Acciones en C/C++ dentro de .l/.y	Visitor/listener en C++ o acciones en la gramática	Lambdas como acciones semánticas

Para el mini-proyecto en C++, **Flex & Bison** (las versiones modernas de Lex & Yacc) son la opción más directa si el curso sigue el enfoque clásico de compiladores. **ANTLR 4** conviene si se quiere el árbol sintáctico ya construido y mejores mensajes de error. **Spirit X3** es útil para lenguajes pequeños sin dependencias externas, pero sus errores de compilación con plantillas son difíciles de leer.

6. Ejercicio: calculadora con Lex & Yacc en C++

Se implementó una calculadora de expresiones aritméticas, basada en el ejemplo de calculadora del Apéndice A del artículo de Yacc [3] y en los ejemplos de Levine [6]. El código C generado se compiló con g++ y las acciones usan std::cout. Se probó con flex 2.6.4 (como lex) y Berkeley Yacc 2.0 (como yacc) en Linux.

Archivo calc.l (analizador léxico):

```
%{
#include <cstdlib>
#define YYSTYPE double    // mismo tipo de valor que en calc.y
extern double yylval;      // valor del token, definido por yacc
#include "y.tab.h"
%}
%%
[0-9]+(\.[0-9]+)? { yylval = atof(yytext); return NUM; }
[-+*/() \n]       { return yytext[0]; }
[ \t]+            { /* ignorar espacios */ }
.                 { return yytext[0]; }
%%
int yywrap() { return 1; } // fin de la entrada
```

Archivo calc.y (analizador sintáctico):

```
%{
#include <iostream>
```

```

#define YYSTYPE double
int yylex();
void yyerror(const char *s) { std::cerr << "Error: " << s << std::endl; }
%}
%token NUM
%left '+' '-'
%left '*' '/'
%right UMENOS
%%
lineas : /* vacio */
        | lineas expr '\n' { std::cout << "=" << $2 << std::endl; }
        | lineas '\n'
        | lineas error '\n' { yyerrok; }
        ;
expr : expr '+' expr { $$ = $1 + $3; }
      | expr '-' expr { $$ = $1 - $3; }
      | expr '*' expr { $$ = $1 * $3; }
      | expr '/' expr { if ($3 == 0) { yyerror("division entre cero"); $$ = 0; }
                        else $$ = $1 / $3; }
      | '-' expr %prec UMENOS { $$ = -$2; }
      | '(' expr ')' { $$ = $2; }
      | NUM { $$ = $1; }
      ;
%%
int main() { return yyparse(); }

```

Compilación y ejecución:

```

yacc -d calc.y          # genera y.tab.c y y.tab.h
lex calc.l              # genera lex.yy.c
g++ -x c++ -o calc y.tab.c lex.yy.c # compila como C++
./calc

```

Los mismos pasos quedan en un Makefile, de modo que basta con ejecutar make y luego ./calc:

```

calc: calc.l calc.y
    yacc -d calc.y
    lex calc.l
    g++ -x c++ -o calc y.tab.c lex.yy.c

clean:
    rm -f calc y.tab.c y.tab.h lex.yy.c

```

Notas de portabilidad (macOS y Linux):

- En macOS, el comando yacc es GNU Bison en modo compatible. Su y.tab.h declara yyval con el tipo YYSTYPE, que por defecto es int. Por eso calc.l define YYSTYPE como double antes de incluir y.tab.h; sin esa línea, la compilación falla con el error *redeclaration of 'yyval' with a different type*.
- La opción -x c++ indica a g++ (que en macOS es clang++) que trate los archivos .c generados como C++, lo que evita la advertencia *treating 'c' input as 'c++'*.
- El ejercicio se compiló y ejecutó con Berkeley Yacc 2.0 y con GNU Bison 3.8 (bison -y), ambos con flex 2.6.4 en Linux. En macOS se usan las herramientas de línea de comandos de Xcode (xcode-select --install).

Resultados obtenidos:

Entrada	Salida	Qué demuestra
$2 + 3 * 4$	$= 14$	* tiene mayor precedencia que + (%left)
$(2 + 3) * 4$	$= 20$	Los paréntesis cambian el orden
$-5 + 10 / 4$	$= -2.5$	Menos unario con %prec y números reales
$7 / 0$	Error: division entre cero	Código propio en una acción
$2 + * 3$	Error: syntax error	Recuperación de errores con el token error
$1.5 * 2$	$= 3$	El análisis continúa tras el error

La gramática de expr es ambigua, pero Yacc resuelve los conflictos *shift/reduce* con las declaraciones %left y %right, sin reescribir la gramática en niveles.

7. Referencias

1. Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.ª ed.). Addison-Wesley. <https://dpvipracollege.ac.in/wp-content/uploads/2023/01/Alfred-V.-Aho-Monica-S.-Lam-Ravi-Sethi-Jeffrey-D.-Ullman-Compilers-Principles-Techniques-and-Tools-Pearson-Addison-Wesley-2007.pdf>
2. Lesk, M. E. y Schmidt, E. (1975). *Lex – A Lexical Analyzer Generator*. Bell Laboratories. <https://cs.utexas.edu/~novak/lexpaper.htm>
3. Johnson, S. C. (1975). *Yacc: Yet Another Compiler-Compiler*. Bell Laboratories. <https://www.cs.utexas.edu/~novak/yaccpaper.htm>
4. The Open Group. *lex – generate programs for lexical tasks*. POSIX.1 (The Open Group Base Specifications). <https://pubs.opengroup.org/onlinepubs/9799919799/utilities/lex.html>
5. The Open Group. *yacc – yet another compiler compiler*. POSIX Programmer's Manual. <https://man.archlinux.org/man/yacc.1p.en>
6. Levine, J. R., Mason, T. y Brown, D. (1992). *lex & yacc* (2.ª ed.). O'Reilly Media. https://d1.amobbs.com/bbs_upload782111/files_33/ourdev_584393GCYRF3.pdf
7. Free Software Foundation. *GNU Bison Manual* (incluye "Conditions for Using Bison"). <https://www.gnu.org/s/bison/manual/bison.html>
8. Flex: The Fast Lexical Analyzer (repositorio oficial). <https://github.com/westes/flex>
9. ANTLR v4 (repositorio oficial y README). <https://github.com/antlr/antlr4> · Sitio oficial: <https://www.antlr.org/index.html>
10. Parr, T., Harwell, S. y Fisher, K. (2014). Adaptive LL(*) Parsing: The Power of Dynamic Analysis. *Proceedings of OOPSLA 2014*, ACM, pp. 579–598. DOI: [10.1145/2660193.2660202](https://doi.org/10.1145/2660193.2660202)
11. ANTLR 4 Documentation – *Grammar Structure*. <https://github.com/antlr/antlr4/blob/master/doc/grammars.md>

12. ANTLR 4 – C++ target / runtime README.
<https://chromium.googlesource.com/external/github.com/antlr/antlr4/+/master/runtime/Cpp>
13. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. The Pragmatic Bookshelf.
<https://dl.icdst.org/pdfs/files3/a91ace57a8c4c8cdd9f1663e1051bf93.pdf>
14. de Guzman, J. y Kaiser, H. *Spirit X3 Documentation*. Boost C++ Libraries.
<https://www.boost.org/latest/libs/spirit/doc/x3/html/index.html>
15. Boost Software License, Version 1.0. http://www.boost.org/LICENSE_1_0.txt
16. Ford, B. (2004). Parsing Expression Grammars: A Recognition-Based Syntactic Foundation. *Proceedings of POPL 2004*, ACM, pp. 111–122. DOI: [10.1145/964001.964011](https://doi.org/10.1145/964001.964011)
17. openSUSE Build Service – especificación del paquete flex (licencia BSD-3-Clause).
<https://build.opensuse.org/projects/openSUSE:Factory:zSystems/packages/flex/files/flex.spec>

8. Anexo

Prompts en Claude Fable 5.1:

Deberás documentar al menos 3 herramientas distintas, donde una de ellas será Lex&Yacc y las otras 2 dame diferentes opciones (principalmente) del enfocado en el lenguaje de C++ porfavor. Aparte hazme un análisis de cada uno explicándome lo siguiente.

- Plataforma y lenguaje base.
- Características básicas (formato requerido en las entradas, tipo de ejecución, etc.)
- Tipo de licenciamiento (costo, si lo tuviera)
- Herramientas teóricas en las que se basan: expresiones regulares, métodos sintácticos empleados, etc.
- Tipo de interfaz.
- Facilidad para inyectar código propio y cómo se haría.

Ayudame dándome las referencias de donde estes sacando la información, para poder visualizarlo

Me gustaría que ahora hicieras ese análisis enfocados en Lex&Yacc, ANTLR 3 y Boost.Spirit X3.

Necesito ayuda para desarrollar un ejercicio con Lex&Yacc usando su documentación porfavor

No lo entendí, podrías explicármelo profavor.

Se corre igual en MacOS que en Linux? Porque ya tengo los Xcode-tools instalados, no se si tenga que instalarlo de nuevo

Me sale este error, que es lo que esta mal?

```
dannywu@DannyWu Tarea2 % xcode-select --install
xcode-select: note: Command line tools are already installed. Use "Software Update" in System Settings
or the softwareupdate command line interface to install updates
dannywu@DannyWu Tarea2 % xcode-select --install
xcode-select: note: Command line tools are already installed. Use "Software Update" in System Settings
or the softwareupdate command line interface to install updates
dannywu@DannyWu Tarea2 %
dannywu@DannyWu Tarea2 % make
yacc -d calc.y
lex calc.l
g++ -o calc y.tab.c lex.yy.c
clang++: warning: treating 'c' input as 'c++' when in C++ mode, this behavior is deprecated [-Wdeprecated]
clang++: warning: treating 'c' input as 'c++' when in C++ mode, this behavior is deprecated [-Wdeprecated]
In file included from calc.l:4:
./y.tab.h:60:16: error: redeclaration of 'yylval' with a different type: 'YYSTYPE' (aka 'int') vs 'double'
60 | extern YYSTYPE yyval;
    | ^
calc.l:3:15: note: previous declaration is here
3 | extern double yyval; // valor del token, definido por yacc
    | ^
1 error generated.
make: *** [calc] Error 1
dannywu@DannyWu Tarea2 % ./calc
zsh: no such file or directory: ./calc
dannywu@DannyWu Tarea2 % ./calc
zsh: no such file or directory: ./calc
dannywu@DannyWu Tarea2 % xcode-select --install
xcode-select: note: Command line tools are already installed. Use "Software Update" in System Settings
or the softwareupdate command line interface to install updates
dannywu@DannyWu Tarea2 % make
yacc -d calc.y
lex calc.l
g++ -o calc y.tab.c lex.yy.c
clang++: warning: treating 'c' input as 'c++' when in C++ mode, this behavior is deprecated [-Wdeprecated]
clang++: warning: treating 'c' input as 'c++' when in C++ mode, this behavior is deprecated [-Wdeprecated]
In file included from calc.l:4:
./y.tab.h:60:16: error: redeclaration of 'yylval' with a different type: 'YYSTYPE' (aka 'int')vs 'double'
60 | extern YYSTYPE yyval;
    | ^
```

```
calc.l:3:15: note: previous declaration is here
3 | extern double yylval; // valor del token, definido por yacc
  | ^
1 error generated.
make: *** [calc] Error 1
dannywu@DannyWu Tarea2 % ./calc
zsh: no such file or directory: ./calc
```

Muchas gracias, por ultimo podrias generar un README.md explicando lo que se hizo en el código porfavor.